



CS61C

Great Ideas
in
Computer Architecture
(a.k.a. Machine Structures)



Assistant
Teaching Professor
Lisa Yan

Parallelism I: Performance Programming

Software

Parallel Requests

Assigned to computer
e.g., Search "Cats"

Parallel Threads

Assigned to core e.g., Lookup, Ads

Parallel Instructions

>1 instruction @ one time
e.g., 5 pipelined instructions

Parallel Data

>1 data item @ one time
e.g., Add of 4 pairs of words

Hardware descriptions

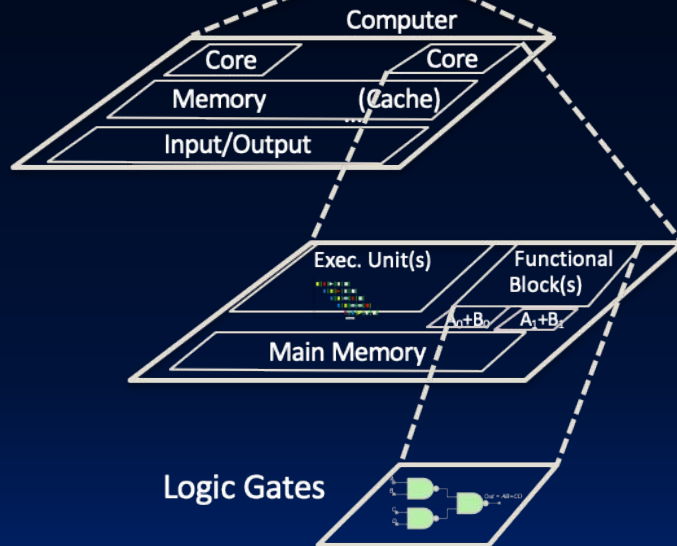
All gates work in parallel,
at same time

Hardware

Warehouse
Scale Computer



PC Smartphone



Don Knuth (1974)

nificant. The conventional wisdom shared by many of today's software engineers calls for ignoring efficiency in the small; but I believe this is simply an overreaction to the abuses they see being practiced by penny-wise-and-pound-foolish programmers, who can't debug or maintain their "optimized" programs. In established engineering dis-



[Paper DOI](#)

[Don Knuth](#)

Turing Award (1974)

Asymptotic Notation,

TeX

There is no doubt that the grail of efficiency leads to abuse. Programmers waste enormous amounts of time thinking about, or worrying about, the speed of noncritical parts of their programs, and these attempts at efficiency actually have a strong negative impact when debugging and maintenance are considered. We *should* forget about small efficiencies, say about 97 % of the time: premature optimization is the root of all evil.

Yet we should not pass up our opportunities in that critical 3 %. A good programmer will not be lulled into complacency by such reasoning, he will be wise to look carefully at the critical code; but only *after* that code has been identified. It is often a mistake to



Amdahl's Law

- Amdahl's Law
- Matrix Multiplication: Performance Benchmark
- Matmul: Non-parallel optimization

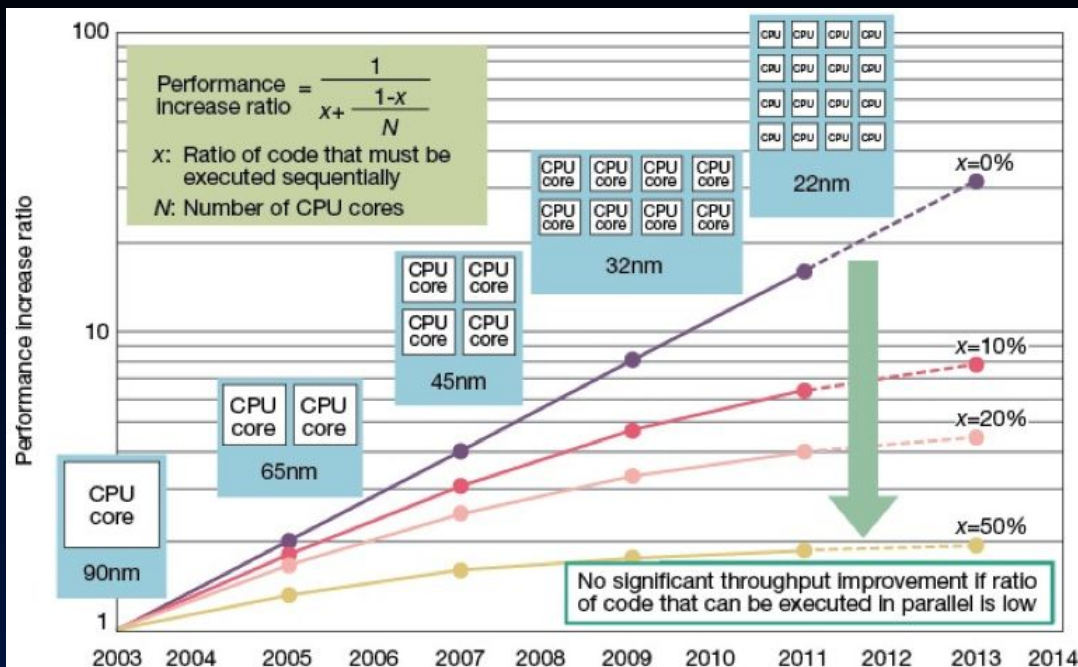
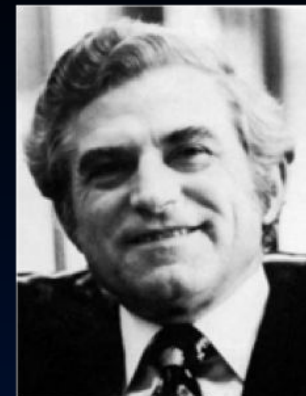


Fig 3 Amdahl's Law an Obstacle to Improved Performance Performance will not rise in the same proportion as the increase in CPU cores. Performance gains are limited by the ratio of software processing that must be executed sequentially. Amdahl's Law is a major obstacle in boosting multicore microprocessor performance. Diagram assumes no overhead in parallel processing. Years shown for design rules based on Intel planned and actual technology. Core count assumed to double for each rule generation.



Gene Amdahl
Computer Pioneer

Performance gains of parallelism are limited by the ratio of software processing that must be executed sequentially.

Amdahl's Law

- A quantitative version of the law of diminishing returns:

$$\text{Speedup with enhancement} = \frac{\text{Exec. time without enhancement}}{\text{Exec. time with enhancement}}$$

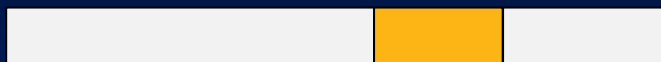
$$\text{Speedup with enhancement} = \frac{1}{\left(1 - \text{frac}_{\text{optimized}}\right) + \frac{\text{frac}_{\text{optimized}}}{\text{factor}_{\text{improvement}}}}$$

Without
enhancement:



$\text{frac}_{\text{optimized}}$

With
enhancement:



$$\text{factor}_{\text{improvement}} = 2$$

$$\text{Speedup with enhancement} = \frac{1}{\left(1 - \text{frac}_{\text{optimized}}\right) + \frac{\text{frac}_{\text{optimized}}}{\text{factor}_{\text{improvement}}}}$$

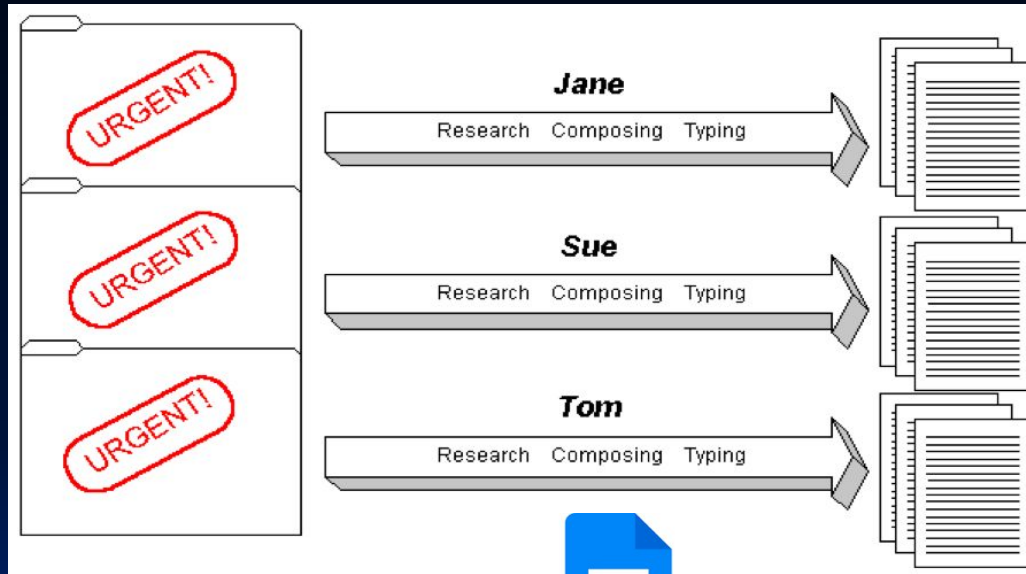
Example: The execution time of **a third of a program** can be accelerated by a **factor of 2**.

- What is the overall program speedup?

$$\begin{aligned}\text{speedup} &= \frac{1}{2/3 + \frac{1/3}{2}} \\ &= 1.2\end{aligned}$$

Performance gains are limited by the component you can improve (with parallelism, micro-optimizations, etc.)

- Reasonable assumption: Even with parallelization, some fraction of a program will need to run sequentially, e.g., to coordinate the parallel work itself.



[Sequential]

Sequential vs Parallel Tradeoff

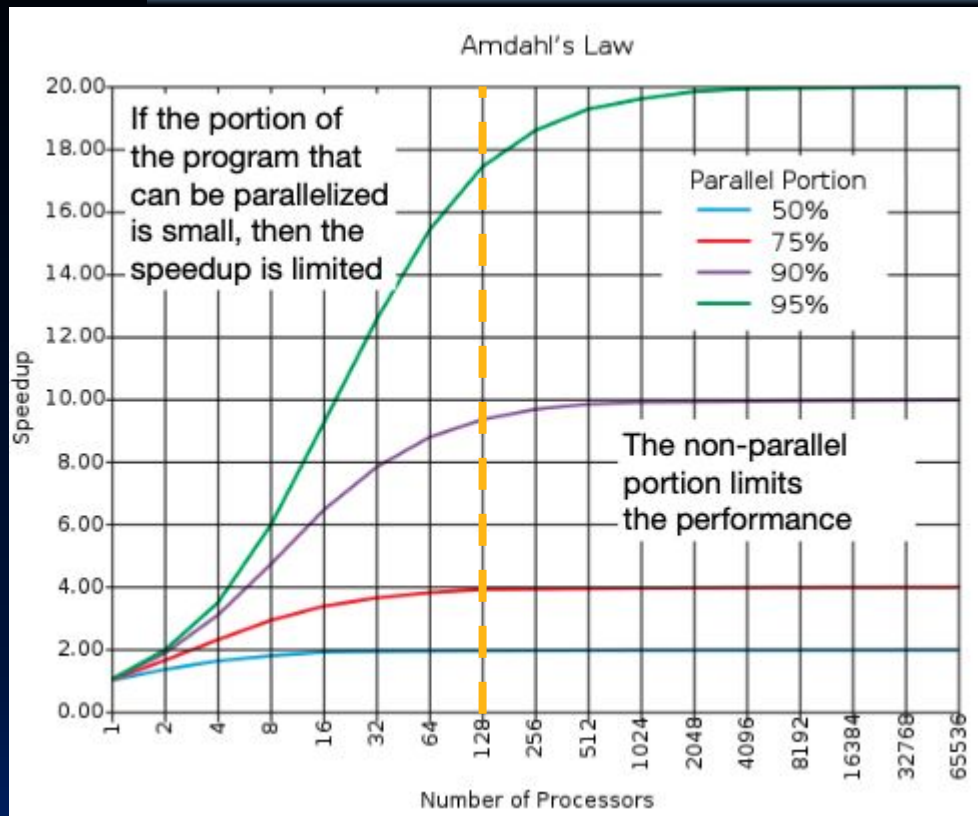
- Suppose we had 128 processors that can improve the parallel component of any program by a **factor of 128**.
- If we are looking to write a program that gets a **10x speedup** from this parallel architecture, what fraction **S** of the program can be **sequential**?

$$\text{Speedup with enhancement} = \frac{1}{\underbrace{(1 - \text{frac}_{\text{optimized}})}_S + \frac{\text{frac}_{\text{optimized}}}{\text{factor}_{\text{improvement}}}}$$

- A. 50%
- B. 15%
- C. 10%
- D. 9%
- E. Something else



Amdahl's Law



D. 9%

Parallelism/HW does not solve everything. You still to choose tasks that actually leverage your architecture!

Example

- Consider an enhancement which runs 20 times faster but which is only usable
 - 25% of the time
 - Speedup w/ E = $1/ (.75 + .25/20) = 1.31$
- What if its usable only 15% of the time?
 - Speedup w/ E = $1/ (.85 + .15/20) = 1.17$
- Amdahl's Law tells us that to achieve linear speedup with 100 processors, none of the original computation can be scalar!
- To get a speedup of 90 from 100 processors, the percentage of the original program that could be scalar would have to be 0.1% or less
- Speedup w/ E = $1/ (.001 + .999/100) = 90.99$



Before we discuss performance improvements
due to parallelism further...

How do we improve program runtime, knowing
what we know?



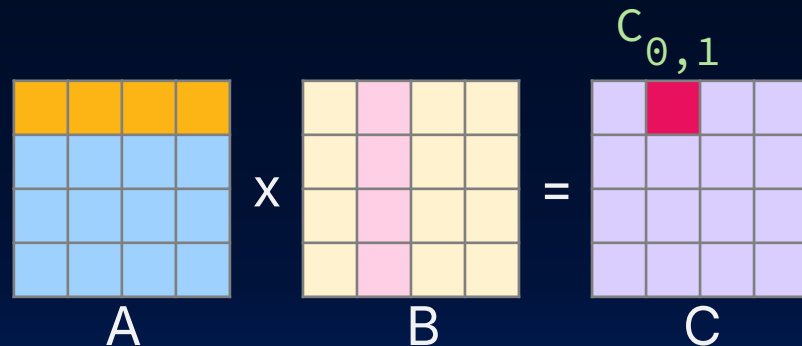
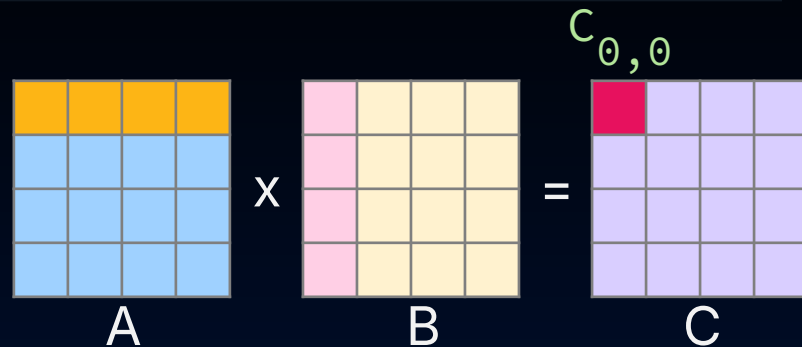
Matrix Multiplication: Performance Benchmark

- Amdahl's Law
- Matrix Multiplication: Performance Benchmark
- Matmul: Non-parallel optimization

Recall Matrix Multiplication

For $A \times B = C$, construct each element of C as follows:

```
def dgemm(N, A, B, C):
    for i in range(N):
        for j in range(N):
            C[i+j*N] = 0
            for k in range(N):
                C[i+j*N] += \
                    A[i+k*N] * B[k+j*N]
```



P&H Figure 2.43
Column-major order
(FORTRAN)

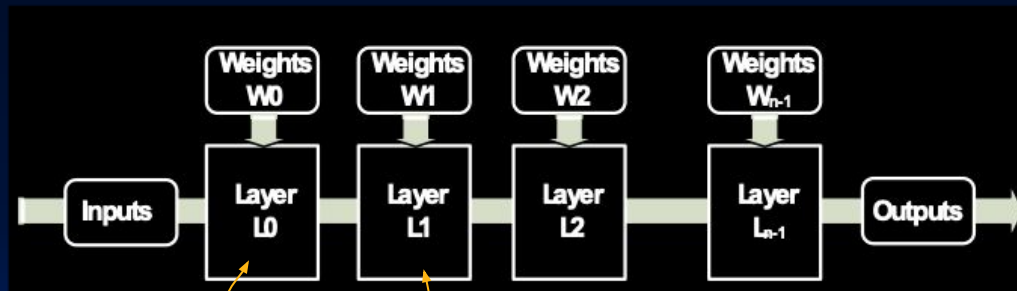
$C_{i,j} : C[i+j*N]$

etc.

Yan, SP26

Matrix Multiplication: Fundamental to many tasks

- Matrix multiplication is a basic operation in many engineering, data, and imaging processing tasks.
 - Image filtering, noise reduction, machine learning...
 - Many closely related operations
- Task (and implementation): **dgemm**
 - Double(-Precision floating-point) GEneral Matrix Multiplication
 - In FORTRAN (which stores matrices in **column order**)



"cat"

Matrix-matrix / matrix-vector multiplications in
each layer of a neural net

30-Parallelism I-Performance Programming(15)



Agenda

matmul: Non-parallel optimization

- Amdahl's Law
- Matrix Multiplication: Performance Benchmark
- Matmul: Non-parallel optimization

- Let $C = A \times B$. A , B , C are $N \times N$ matrices in column-major order.
- **FLOPS**: Floating Point Operations Per Second.
 - dgemm has $2 \times N^3$ Floating Point Operations (**fadd**, **fmul**)

```
def dgemm(N, A, B, C):
    for i in range(N):
        for j in range(N):
            C[i+j*N] = 0
            for k in range(N):
                C[i+j*N] += \
                    A[i+k*N] * B[k+j*N]
```

N	Python (MFLOPS)	Python (GFLOPS)
32	5.4	0.0054
160	5.5	0.0055
480	5.4	0.0054
960	5.3	0.0053

*To calculate FLOPS, measure runtime, scale by # Floating Point Operations in program, and invert.



C dgemm

- Let $C = A \times B$, where A, B, C are $N \times N$ matrices in column-major order.
- FLOPs**: Floating Point Operations Per Second.
 - dgemm has $2 \times N^3$ Floating Point Operations (**fadd, fmul**)

```
void dgemm_scalar(int N, double *A,
                  double *B, double *C) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            double Cij = 0;
            for (int k = 0; k < N; k++) {
                Cij += \
                    A[i+k*N] * B[k+j*N];
            }
            C[i+j*N] = Cij;
        }
    }
}
```

N	Python (MFLOPs)	Python (GFLOPs)	C (GFLOPs)
32	5.4	0.0054	1.30
160	5.5	0.0055	1.30
480	5.4	0.0054	1.32
960	5.3	0.0053	0.91

Just by switching to C, we get a huge performance boost!

We could stop here...but let's do better!!

P&H Figure 2.43

Column-major order

Timing Program Execution: Lecture Code

- Example lecture code in Drive. Notes/caveats:
 - Naive C, Python use **row-major order**
 - Both C and Python allocate heap memory in matmul
 - Assume square matrices for simplicity

```
def matmul(A, B, N):
    C = [0 for _ in A]
    for i in range(N):
        for j in range(N):
            C[i*N+j] = 0
            for k in range(N):
                C[i*N+j] += \
                    A[i+k*N] * B[k+j*N]
```

$C_{i,j} : C[i*N+j]$



Run Demo

Download `lec30_code.zip` from Lecture Drive

Then, on local:

```
$ scp lec30_code.zip hiveNN:~/.
```

On hive:

```
$ unzip lec30_code.zip
```

```
$ cd lec30_code/
```

```
$ make
```

```
$ ./compareruntimes.sh 100 # 100 x 100 matrices
```

```
$ # try 100, 300, 500
```

Check individual files



Caveat: Hive runtime

- Timing libraries measure **execution time**, not runtime.
 - Big difference in **shared computer environments** / heavy system load (e.g., hive)

```
#include <time.h>
int main(...) {
    clock_t start = clock();
    // ... task code ...
    clock_t end = clock();

    // execution time in seconds
    double delta_time = (double) (end - start)/CLOCKS_PER_SEC;
    ...
}
```



Discuss

1. `matmul.c`: Are matrices row-major order or column-major? How do you know?
2. `matmul_opt.c`: What does the `register` keyword do?
3. `matmul_opt_transpose.c`: Why might this be faster?
4. `matmul_opt_transpose_float.c`: Why might this be slower?
5. `matmul_np.py`: Why is this so fast?



Yan, SP26



Summary

- Python vs. C
- Registers
- Cache blocking
- int vs. float
- [lab] Loop unrolling

See reference slides! (and notes, coming soon)

Two Observations, Made in Parallel

- Element-wise multiplication (of row of A, column of B) can be computed independently
 - One FP instruction: element-wise multiplication
- Different elements of matrix C can be computed independently
 - Separate sequences of fmul + fadd

```
def matmul(A, B, N):  
    C = [0 for _ in A]  
    for i in range(N):  
        for j in range(N):  
            C[i*N+j] = 0  
            for k in range(N):  
                C[i*N+j] += \  
                    A[i+k*N] * B[k+j*N]
```

Perhaps...parallelism?



[Reference] Single-Thread Optimizations of dgemm

- Amdahl's Law
- Matrix Multiplication: Performance Benchmark
- Matmul: Non-parallel optimization

Optimization 1: Loop Unrolling and Function Inlining

- Main idea: Since branches and jumps take a long time, reduce the number of jumps/branches needed to run a program
- Function inlining: Replace a function call with the body of that function
- Loop unrolling: Increase the number of steps done per iteration of a loop

```

int f(int i) {return i*i;}
for(int i = 0; i < max; i++)
{
    arr[i] = f(i);
}

int i = 0;
for(; i < max/4*4; i+=4) {
    arr[i] = i * i;
    arr[i+1] = (i+1) *
    (i+1);
    arr[i+2] = (i+2) *
    (i+2);
    arr[i+3] = (i+3) *

```



Optimization 1: Loop Unrolling and Function Inlining

- Function inlining:
 - Significant reductions in runtime due to not having to set up the stack, deal with calling convention, etc.
 - Can also request the compiler inline a function for you with the `inline` keyword; i.e. `inline int f(int i)`
- Loop unrolling: Increase the number of steps done per iteration of a loop
 - Reduces runtime due to fewer branches, and can have a surprisingly large impact
 - Often done automatically by the gcc compiler, but manually unrolling can improve runtime over the automated system

Loop Unrolling and Function Inlining Limitations

- Generally causes your code file/resulting executable to be larger
- If your loop is too large, you end up exceeding your branch immediate, and take a large penalty to runtime
- The `inline` keyword is only a *suggestion* to the compiler; gcc is perfectly free to ignore the keyword entirely.
- Loop unrolling is already done by some compilers, so speedup may be minimal/hard to predict
- Major: Loop unrolling makes your code much harder to read and modify. Thus it's generally best to only unroll at the end, or to save a rolled version of your code for later use.

Optimization 2: Variable Caching

- Main idea: Save commonly used values in variables, instead of forcing recomputations
- Additionally, save commonly used variables in registers, instead of saving them on the stack. The `register` keyword can be used to request a variable gets put in a register

- **x86 is the assembly language commonly used by PCs, and has fewer registers than RISC-V does. Thus, most variables get stored in the stack, instead of in registers.**

```

for(int i = 0; i < max/4; i++)
{
    f(i);
}

register int i = 0;
register int maxoverfour =
max/4;
for(; i < maxoverfour;
i++) {
    f(i);
}
    
```



Variable Caching Limitations

- Limited by the number of registers available
- Since different CPUs can have different registers, this makes your performance more dependent on the architecture than before, which means:
 - You need to know how the architecture works to optimize your code
 - Your code will work slower on a different architecture
- As before, the compiler tends to do some of this already, so getting speedup can be inconsistent. The `register` keyword is also only a suggestion, and the compiler is free to ignore the keyword.

Optimization 3: Registers, Cache Blocking

- Variable caching: Save commonly used variables in registers
- Accessing main memory takes hundreds of clock cycles
- Data caching: Save commonly used data "closer" to the CPU, so we can access it in fewer cycles (~10 cycles)
- Even better: Predict what data you need and bring that data closer to the CPU before we even access it
- For today: accessing adjacent memory addresses will be on average faster than accessing nonadjacent memory addresses

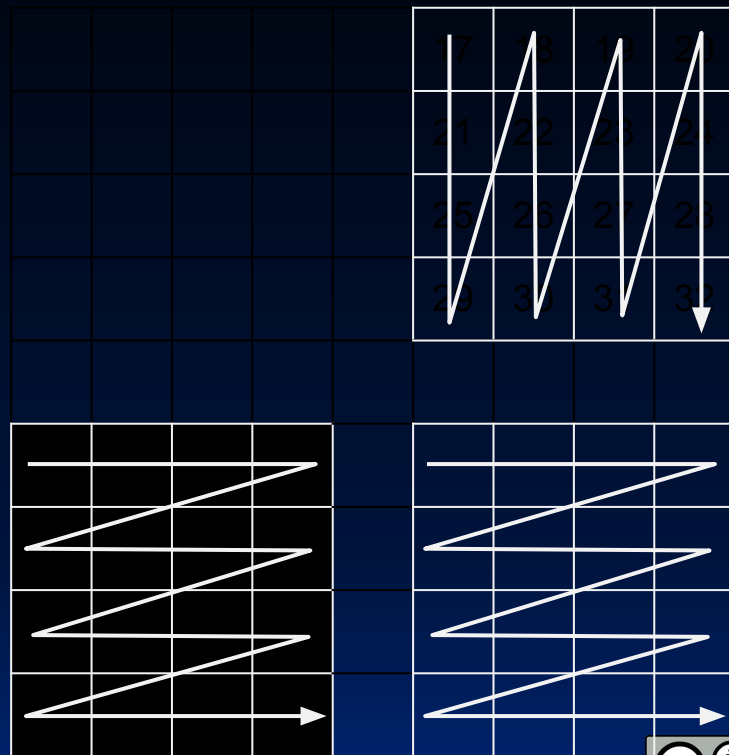


Data Caching Example

```
Test 91: 0.018000 seconds for adjacent, 0.187000 seconds for nonadjacent
Test 92: 0.015000 seconds for adjacent, 0.138000 seconds for nonadjacent
Test 93: 0.017000 seconds for adjacent, 0.168000 seconds for nonadjacent
Test 94: 0.034000 seconds for adjacent, 0.240000 seconds for nonadjacent
Test 95: 0.015000 seconds for adjacent, 0.274000 seconds for nonadjacent
Test 96: 0.019000 seconds for adjacent, 0.167000 seconds for nonadjacent
Test 97: 0.009000 seconds for adjacent, 0.143000 seconds for nonadjacent
Test 98: 0.011000 seconds for adjacent, 0.173000 seconds for nonadjacent
Test 99: 0.014000 seconds for adjacent, 0.221000 seconds for nonadjacent
Average adjacent test time: 0.016570 seconds
Average nonadjacent test time: 0.168620 seconds
```

Cache Blocking for Matrix Multiplication

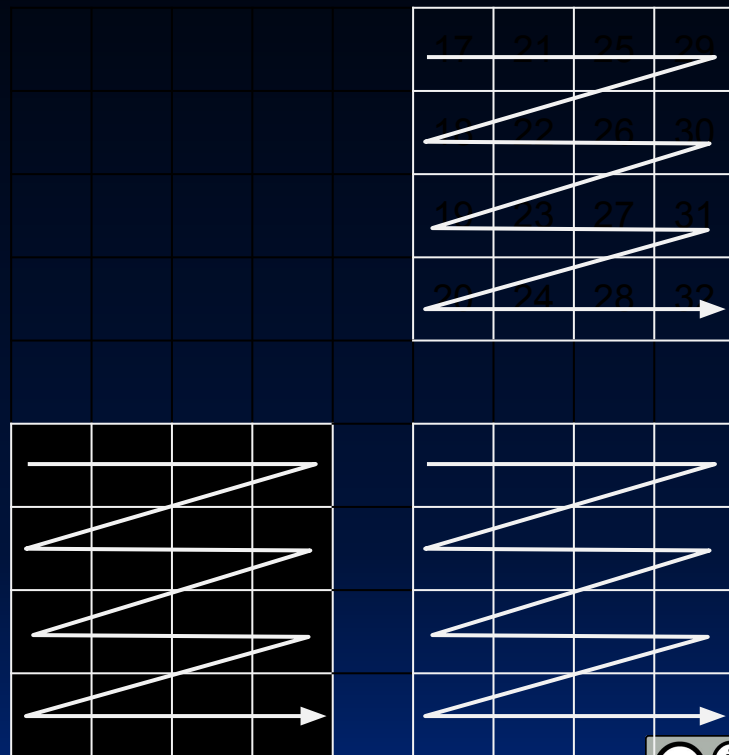
- Assuming we do a naive matrix multiplication, data gets accessed in row-major order for two matrices, and column-major order in one matrix
- Thus, really fast to access two matrices, really slow to access the third.
- Optimization: Transpose the third matrix before starting calculations, so we get fast accesses on all matrices



Yan, SP26

Cache Blocking for Matrix Multiplication

- Assuming we do a naive matrix multiplication, data gets accessed in row-major order for two matrices, and column-major order in one matrix
- Thus, really fast to access two matrices, really slow to access the third.
- Optimization: Transpose the third matrix before starting calculations, so we get fast accesses on all matrices



Yan, SP26



Cache Blocking for Matrix Multiplication

- Optimization: Transpose the third matrix before starting calculations, so we get fast accesses on all matrices
- Transposing adds $\Theta(N^2)$ slow memory accesses, but converts $\Theta(N^3)$ slow memory accesses into fast memory accesses. Thus, this should be faster for large matrices, but slower for small matrices.
- Any way to get the best of both worlds?
 - Decide on a threshold; if the size of the matrix is smaller than the threshold, use the naive approach, and if the size of the matrix is larger than the threshold, use the transpose approach
- How to decide threshold?
 - Run tests on a representative machine to find where the break-even point is.